Quantum Computing

Fault-Tolerant Quantum Computing

Lecture 9 Notes

Ralph Razzouk Marc Fares Joseph Jabbour

Lebanese Science Organization

Prepared for the LebNet and LebSci Quantum Computing Course 2026

About these notes

These notes accompany a nine-lecture course on quantum computing designed and produced by the Lebanese Science Organization. Each chapter corresponds to one lecture. The lectures are concept-focused: they explain what a quantum state is, how entanglement, interference, and measurement combine to produce computation, and what is required to make that computation reliable. Linear algebra, complex numbers, and elementary probability are assumed.

Each chapter names its lecturer and date, opens with a short preface, develops its material with the derivations in place, and closes with a summary. Worked examples appear in numbered boxes, as do definitions, theorems, remarks, and common misconceptions. Works cited in the text are collected in a single reference list at the end of the document.

Conventions used throughout: qubits in a register are labelled from left to right, so that in $|ab\rangle$ the first qubit is a; the first qubit is the top wire in every circuit diagram; the computational basis of two qubits is ordered $|00\rangle, |01\rangle, |10\rangle, |11\rangle$; circuits are read from left to right, so a gate U_1 followed by a gate U_2 acts as the matrix product U_2U_1.

This chapter is exported individually from the full lecture notes (9 of 9). These notes are made available for open, public study. Reuse conditions will be stated separately by the Lebanese Science Organization.

Contents

1 Fault-Tolerant Quantum Computing **3**
1.1 Faults and error propagation . . . 3
1.1.1 Error propagation . . . 4
1.1.2 Fault amplification . . . 5
1.2 The fault-tolerance requirement . . . 6
1.3 The threshold theorem . . . 8
1.3.1 Concatenation overhead . . . 8
1.3.2 Statement and assumptions . . . 8
1.3.3 Threshold estimates . . . 9
1.3.4 Surface code overhead . . . 10
1.4 Summary . . . 11

References **12**

Chapter 1

Fault-Tolerant Quantum Computing

Lecturer: Ralph Razzouk

Date: November 21, 2026

?? showed that a code protects stored information against a single error, provided that encoding, syndrome measurement, and recovery are themselves perfect. In a real device the components that perform the correction fail at rates comparable to the data they protect, and a two-qubit gate can turn one error into two. This lecture asks whether a computation built entirely from faulty components can be made reliable, and at what cost. It traces how errors propagate through gates, states the fault-tolerance requirement and the design principles that meet it, and closes with the threshold theorem and the overhead it demands.

1.1 Faults and error propagation

In **??** the three-qubit bit-flip code $\alpha|000\rangle + \beta|111\rangle$ reduced the probability of an uncorrectable bit flip from p to about $3p^2$. The argument assumed that the encoding circuit is perfect, that the parity checks Z_1Z_2 and Z_2Z_3 are measured perfectly, and that the recovery is applied perfectly; only the stored data qubits were allowed to fail. This lecture removes those assumptions and asks what survives.

Definition 1.1: Fault and error

A location in a circuit is one elementary operation: a gate, a qubit preparation, a measurement, or a qubit left idle for one time step. A fault is a location whose physical operation deviates from its ideal. We model a fault as the ideal operation followed by a Pauli error, X, Z, or $Y = \mathrm{i}XZ$, on each qubit the location touches, and we assume that each location is faulty independently with probability p. An error is the resulting deviation of a data qubit from its intended state. One fault may produce several errors.

The distinction matters because the code counts errors, not faults. If every fault produced exactly one error, a circuit with L locations would fail only when two of them are faulty, with probability of order $\binom{L}{2}p^2$, which is the quadratic suppression that **??** promised. If a single fault can produce two errors in the same block, then for a code that corrects one error the failure probability is of order p, and the code has bought nothing.

Removing the first assumption is harmless by itself: if gates fail with probability p but the resulting errors stay where they were born, an encoded operation of n gates has n chances to acquire one correctable error. The difficulty lies with the other two assumptions. An ancilla used to measure

a parity check touches several data qubits in succession, and an error on it may flow onto any of them. A measurement may report the wrong syndrome, in which case the recovery applies a "correction" to a healthy qubit and is itself the source of an error. To make this precise we need to know how errors move through gates.

1.1.1 Error propagation

Definition 1.2: Error propagation

Let U be a unitary gate and E a Pauli error on some of its qubits just before it. Since $UE = (UEU^\dagger)\,U$, the faulty sequence equals the ideal gate followed by the error $UEU^\dagger$. We say that E propagates through U to $UEU^\dagger$, and that it has spread if $UEU^\dagger$ acts on more qubits than E did.

A single-qubit gate cannot spread anything. For the CNOT of **??**, control on the first wire and target on the second, the four basic cases are these.

Proposition 1.1 Propagation of Pauli errors through CNOT

With $\mathrm{CNOT} = |0\rangle\langle 0| \otimes \mathbb{I} + |1\rangle\langle 1| \otimes X$ and $\mathrm{CNOT}^2 = \mathbb{I}$,

$$\mathrm{CNOT}\,(X \otimes \mathbb{I})\,\mathrm{CNOT} = X \otimes X, \qquad \mathrm{CNOT}\,(\mathbb{I} \otimes X)\,\mathrm{CNOT} = \mathbb{I} \otimes X, \tag{1.1}$$

$$\mathrm{CNOT}\,(Z \otimes \mathbb{I})\,\mathrm{CNOT} = Z \otimes \mathbb{I}, \qquad \mathrm{CNOT}\,(\mathbb{I} \otimes Z)\,\mathrm{CNOT} = Z \otimes Z. \tag{1.2}$$

An X error on the control is copied onto the target; a Z error on the target is copied onto the control; the other two stay where they are.

Each identity is a one-line computation on basis states, with $\oplus$ denoting addition modulo two.

Proof. Write $\mathrm{CNOT}\,|a,b\rangle = |a, a \oplus b\rangle$ for $a, b \in \{0,1\}$, with $X\,|a\rangle = |a \oplus 1\rangle$ and $Z\,|a\rangle = (-1)^a\,|a\rangle$. Since $\mathrm{CNOT}^2 = \mathbb{I}$, the operator $\mathrm{CNOT}\,E\,\mathrm{CNOT}$ is the propagated error of Definition 1.2. On $|a,b\rangle$,

$$\mathrm{CNOT}\,(X \otimes \mathbb{I})\,\mathrm{CNOT}\,|a,b\rangle = \mathrm{CNOT}\,|a \oplus 1, a \oplus b\rangle = |a \oplus 1, b \oplus 1\rangle = (X \otimes X)\,|a,b\rangle\,, \tag{1.3}$$

$$\mathrm{CNOT}\,(\mathbb{I} \otimes X)\,\mathrm{CNOT}\,|a,b\rangle = \mathrm{CNOT}\,|a, a \oplus b \oplus 1\rangle = |a, b \oplus 1\rangle = (\mathbb{I} \otimes X)\,|a,b\rangle\,, \tag{1.4}$$

$$\mathrm{CNOT}\,(Z \otimes \mathbb{I})\,\mathrm{CNOT}\,|a,b\rangle = (-1)^a\,\mathrm{CNOT}\,|a, a \oplus b\rangle = (-1)^a\,|a,b\rangle = (Z \otimes \mathbb{I})\,|a,b\rangle\,, \tag{1.5}$$

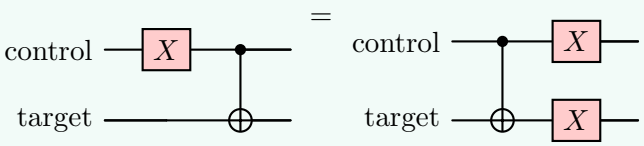
$$\mathrm{CNOT}\,(\mathbb{I} \otimes Z)\,\mathrm{CNOT}\,|a,b\rangle = (-1)^{a \oplus b}\,\mathrm{CNOT}\,|a, a \oplus b\rangle = (-1)^{a+b}\,|a,b\rangle = (Z \otimes Z)\,|a,b\rangle\,, \tag{1.6}$$

where $(-1)^{a \oplus b} = (-1)^{a+b}$ because both depend only on the parity of $a + b$. The four operators agree with the claimed products on a basis, so they are equal, which proves Proposition 1.1. Multiplying the first line on the right by CNOT gives $\mathrm{CNOT}\,(X \otimes \mathbb{I}) = (X \otimes X)\,\mathrm{CNOT}$, the identity of Example 1.1 below. Every two-qubit Pauli is a product of these four up to a phase, so every Pauli error propagates through CNOT to a Pauli error; for instance $Y \otimes \mathbb{I} = \mathrm{i}\,(X \otimes \mathbb{I})(Z \otimes \mathbb{I})$ propagates to $\mathrm{i}\,(X \otimes X)(Z \otimes \mathbb{I}) = Y \otimes X$. ∎

Bit flips travel forward from control to target and phase flips travel backward from target to control, another appearance of the exchange between bit value and phase that H performs in **??**.

Example 1.1 (Spread of a control error)

The two circuits below are equal as operators. A shaded box is not a gate we apply; it marks a fault. On the left, an X fault occurs on the control just before the CNOT; on the right, the CNOT acts ideally and is followed by X on both wires.



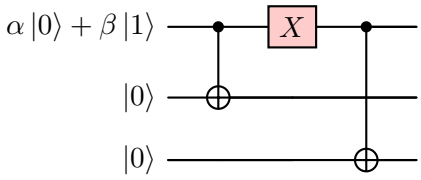
In symbols, $\mathrm{CNOT}\,(X \otimes \mathbb{I}) = (X \otimes X)\,\mathrm{CNOT}$, the first identity of Proposition 1.1 multiplied on the right by CNOT. One fault has become two errors.

The point of the example is that a two-qubit gate is a channel along which errors travel, so a circuit that is correct when every component is ideal may double its errors when one component is not.

1.1.2 Fault amplification

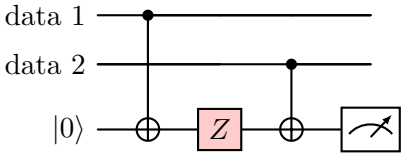
Now return to the extraction circuit of **??**: an ancilla in $|0\rangle$ is the target of a CNOT from data qubit 1 and then of a CNOT from data qubit 2, and is measured. Bit-flip errors on the data flow forward into the ancilla, which is what a parity measurement should do. But by Proposition 1.1 a Z error on the ancilla flows backward onto every data qubit whose gate comes after it. Suppose a Z fault occurs on the ancilla between the two gates: it sits on the target of the second CNOT and propagates to Z_2 on the data, a phase error that the bit-flip code cannot see, so the ancilla, which was supposed to observe the data, has written on it, and the circuit has introduced a failure at first order in p. This case is worked out below, together with the others: a fault before both gates, checks that touch four qubits, and an X fault in the encoding circuit.

Forward propagation. The three-qubit code is encoded by two CNOT gates from the first qubit, mapping $(\alpha\,|0\rangle + \beta\,|1\rangle)\,|00\rangle$ to $\alpha\,|000\rangle + \beta\,|111\rangle$. Suppose an X fault occurs on the first qubit between the gates.



After the first gate the state is $\alpha\,|000\rangle + \beta\,|110\rangle$; the fault gives $\alpha\,|100\rangle + \beta\,|010\rangle$; the second gate gives $\alpha\,|101\rangle + \beta\,|010\rangle = X_1X_3\,(\alpha\,|000\rangle + \beta\,|111\rangle)$, as Proposition 1.1 predicts. The checks read $Z_1Z_2 = -1$ and $Z_2Z_3 = -1$, since qubit 2 disagrees with both neighbours, and the syndrome table of **??** attributes this to a single error on qubit 2. The recovery applies X_2 and the state becomes $\alpha\,|111\rangle + \beta\,|000\rangle$, a logical bit flip. One fault of probability p has caused a logical failure, so the encoded qubit fails at order p rather than $3p^2$. Reordering the gates does not help: a fault before both gates produces $X_1X_2X_3$ directly. A fault-tolerant preparation instead verifies the encoded state by measuring its checks with the methods of section 1.2 and discarding it if any check fails.

Backward propagation. The circuit for Z_1Z_2 prepares an ancilla in $|0\rangle$, applies CNOT from data qubit 1 and then from data qubit 2 into it, and measures it, obtaining $a_1 \oplus a_2$. Suppose a Z fault occurs on the ancilla between the two gates.



The fault sits on the target of the second gate, so by the fourth identity of Proposition 1.1 it propagates to Z on the ancilla and on data qubit 2. The Z on the ancilla is invisible to a computational-basis measurement, so the syndrome is reported correctly, but data qubit 2 now carries an error the circuit introduced: $Z_2(\alpha|000\rangle + \beta|111\rangle) = \alpha|000\rangle - \beta|111\rangle$ is a logical phase error, undetected by either check. A fault before both gates would propagate to $Z_1 Z_2$, which is the check itself and leaves every codeword unchanged. In general, if one ancilla is coupled to w data qubits in sequence and a Z fault occurs after the k-th gate, the error lands on the last $w - k$ data qubits. For $w = 2$ this is a harmless stabilizer or a single error; for the four-qubit checks of the seven-qubit Steane code, $k = 1$ produces a weight-three error that equals a single error multiplied by the check itself and is therefore corrected, while $k = 2$ produces a weight-two error which a distance-three code misidentifies exactly as in the encoding example, and the correction round fails at order p. The surface code also has four-qubit checks measured through a single ancilla, and the same weight-two error, there called a hook error, does occur; it is rendered harmless by choosing the order of the four gates so that the error lies along a direction that does not shorten any logical operator. Shor-style cat-state ancillas and Steane-style encoded ancillas prevent the problem for general codes by giving each data qubit its own ancilla qubit, so that $w - k$ is at most one.

> **Remark 1.1** Correction as a source of failure
>
> The correction procedure was meant to lower the failure probability from order p to order p^2. If it contains a location where one fault becomes two errors, or where one fault makes the recovery act on healthy data, then the procedure fails with probability of order p and the code has bought nothing. Whether a correction circuit helps or harms is a property of the circuit, not of the code.

1.2 The fault-tolerance requirement

The three-qubit code corrects one error per block. The requirement on the circuits that manipulate encoded data is therefore that no single fault, wherever it occurs, produce more than one error in any block.

Definition 1.3: Fault-tolerant component

An encoded operation, meaning an encoded gate, state preparation, measurement, or round of error correction, is fault tolerant with respect to a code that corrects one error if a single fault at any location inside it produces at most one error on each encoded block at its output. For a code correcting t errors, any $s \leqslant t$ faults must produce at most s errors per block. The requirement concerns the propagation of errors within the component, not their absence.

A fault-tolerant component fails at every location with probability p, like anything else. What the definition guarantees is that when it fails once, the damage is still within the power of the code to repair. Four design principles, used together, achieve this.

Definition 1.4: Transversal gate

Let a logical qubit be stored in a block of n physical qubits. An encoded operation is transversal if it is a tensor product of gates each acting on at most one qubit of each block: a single-block gate $U_1 \otimes \cdots \otimes U_n$ with one factor per qubit, or a two-block gate that couples the i-th qubit of one block only to the i-th qubit of the other.

A transversal gate is automatically fault tolerant, since each elementary gate touches one qubit per block and Proposition 1.1 cannot carry an error between two qubits that no gate connects.

The logical X of the three-qubit code is $X \otimes X \otimes X$, and the logical CNOT between two blocks is CNOT on each of the three corresponding pairs; both are transversal. Which gates are transversal depends on the code.

The second principle is verified ancilla preparation: an ancilla is checked, for instance by measuring a parity of its qubits, and discarded if the check fails, so that a faulty ancilla is caught before it can write on the data. Preparation becomes probabilistic and consumes more ancillas, which the threshold theorem assumes are available in fresh supply. The third is repeated syndrome extraction: because a measurement can lie, the syndrome is measured several times and a recovery is applied only when the readings agree, by majority or by repetition until two consecutive rounds coincide, so that a single measurement fault becomes an outvoted minority rather than a wrong recovery. The cost is time, during which the data continue to accumulate errors. The fourth is that no single ancilla qubit may touch several data qubits of the same block. Shor-style extraction [1] uses a verified cat state $(|0\cdots0\rangle + |1\cdots1\rangle)/\sqrt{2}$ with one ancilla qubit per data qubit in the check; Steane-style extraction [2] uses an entire encoded block as the ancilla, coupled to the data by a transversal CNOT and then measured qubit by qubit. We name both without constructing them. Each gives the ancilla the same connectivity as the data, so that a fault on it reaches at most one data qubit.

Wrong Concept 1.1: A code alone makes a fault-tolerant computer

A code is a subspace together with a set of checks, and it determines which error patterns can be told apart. Fault tolerance is a property of the circuits that encode, check, correct, and compute on that subspace. The same three-qubit code protects a stored qubit when its checks are extracted with verified ancillas and repeated measurements, and fails at first order in p when they are extracted with the naive circuit of **??**. Having a code is necessary for reliable computation; it is not sufficient, and most of the engineering of a quantum computer lies in the difference.

Remark 1.2 Transversal gates are not universal

No code admits a transversal implementation of a universal gate set such as $\{H, T, \mathrm{CNOT}\}$ from **??**. Eastin and Knill proved in 2009 that the transversal gates of any code that detects at least one error form a set that cannot approximate every unitary [3]. Typically H and CNOT are transversal and T is not. The missing gate is supplied by magic-state distillation, in which many noisy copies of a non-stabilizer state are consumed to produce one clean copy that enacts T through measurement, or by code switching, in which the data are moved temporarily into a second code whose transversal set contains the missing gate. We name these without constructing them; in current architectures they account for a large share of the total qubit count.

1.3 The threshold theorem

Suppose every encoded operation is fault tolerant in the sense of Definition 1.3, using a code that corrects one error. A single fault is repaired by the next round of correction, so a logical failure requires two faults in the same operation. If the operation has L locations, this has probability at most $\binom{L}{2}p^2 = c\,p^2$ for a constant c fixed by the size of the fault-tolerant circuit. For $p < 1/c$ encoding has helped; for $p > 1/c$ the many gates of the correction circuit introduce more errors than they remove.

Concatenation repeats the improvement. Each physical qubit of the code is replaced by an encoded block, so n qubits become n^2, and each physical gate of the level-one circuit is replaced by a level-one encoded gate. The level-two operation sees components that fail with probability $p_1 = cp^2$ and itself fails with probability $p_2 = cp_1^2$. After k levels,

$$p_k = c\,p_{k-1}^2, \qquad p_0 = p, \qquad \text{so that} \qquad p_k = \frac{(c\,p)^{2^k}}{c}, \tag{1.7}$$

as section 1.3.1 verifies. If $cp < 1$ the logical failure probability falls doubly exponentially in k while the qubit count per logical qubit grows only as n^k. The value $1/c$ at which the recursion turns from contracting to expanding is the threshold.

1.3.1 Concatenation overhead

Multiplying $p_k = c\,p_{k-1}^2$ by c gives $cp_k = (cp_{k-1})^2$, so $q_k = cp_k$ satisfies $q_k = q_{k-1}^2 = q_0^{2^k} = (cp)^{2^k}$ and $p_k = (cp)^{2^k}/c$, which is (1.7). If $cp < 1$ then $p_k \to 0$; if $cp > 1$ then p_k grows; the boundary $p = 1/c$ is the threshold of the concatenated scheme. To reach a target ε we need $(cp)^{2^k} \leqslant c\varepsilon$, that is

$$k \geqslant \log_2 \frac{\log(1/(c\varepsilon))}{\log(1/(cp))}, \qquad \text{giving the qubit overhead} \qquad n^k = \left(\frac{\log(1/(c\varepsilon))}{\log(1/(cp))}\right)^{\log_2 n}, \tag{1.8}$$

a power of $\log(1/\varepsilon)$, that is, polylogarithmic in the target precision. The gate overhead g^k, with g the number of locations in one fault-tolerant encoded gate, has the same form. A circuit of N gates needs each encoded gate accurate to about ε/N, which replaces ε by ε/N and gives the factor polynomial in $\log(N/\varepsilon)$ of Theorem 1.1. The constant c counts pairs of locations in a fault-tolerant gate that can together cause failure; for the seven-qubit code with Shor-style extraction it is of order 10^3 to 10^4, so that early estimates of the threshold were of order 10^{-4} to 10^{-3}, with the rigorous proofs giving smaller values still.

1.3.2 Statement and assumptions

Definition 1.5: Threshold

Fix a fault-tolerant scheme, meaning a code family or concatenation procedure together with fault-tolerant circuits for a universal gate set, and fix a noise model. The threshold p_{th} of the scheme is the largest physical error rate such that for every $p < p_{\text{th}}$ the logical error rate can be made as small as desired by increasing the size of the code. Above it, increasing the code size does not help and eventually hurts.

Theorem 1.1 Threshold theorem, informal statement

Consider a noise model in which each location is faulty independently with probability p, or in which correlations between faults decay rapidly. Suppose that fresh ancilla qubits in a

known state can be supplied at any time, that gates may be applied in parallel to disjoint sets of qubits, and that the classical processing of syndromes is fast and reliable. Then there is a threshold $p_{\text{th}} > 0$ such that for every $p < p_{\text{th}}$, any ideal circuit with N locations can be simulated to within any accuracy ε by a fault-tolerant circuit whose size exceeds N by a factor polynomial in $\log(N/\varepsilon)$. The theorem is due to Aharonov and Ben-Or in 1997 [4], to Knill, Laflamme, and Zurek in 1998 [5], and to Kitaev in 1997 [6], building on the constructions of Shor in 1996 [1]. The proof is not part of this course.

Each assumption marks a way the argument can fail. Fresh ancillas are needed because correction removes from the data the randomness introduced by errors and must put it somewhere. Parallelism is needed because qubits waiting their turn for a single gate accumulate errors faster than correction removes them. Fast classical processing is needed because the recovery cannot be chosen until the syndrome is decoded. Independence is needed because the argument counts pairs of faults; a process that disturbs many qubits at once, such as a burst of radiation or a drift in a control parameter shared by every gate, is not covered by the statement.

The theorem is a statement about two kinds of qubits. A physical qubit is one of the two-level systems in the device; a logical qubit is the encoded qubit, stored across many physical qubits and manipulated by fault-tolerant encoded operations. The algorithms of **??????** are written for logical qubits, and the theorem converts them into larger circuits on physical qubits with the same output distribution up to ε. The conversion is not free. For the surface code [7, 8], the code family currently regarded as the leading candidate for large devices, which places physical qubits on a two-dimensional lattice with checks between neighbours and which we name but do not construct, the cost is set by the distance of a code, meaning the smallest number of single-qubit errors that maps one codeword to another, so that a code of distance d corrects up to $(d-1)/2$ errors. At a physical error rate a tenth of its threshold, distances between roughly 15 and 30 are needed for the logical error rates that long computations require, which is several hundred to about two thousand physical qubits per logical qubit before counting the qubits spent on magic-state distillation; the noise-model assumptions behind these orders of magnitude are in section 1.3.3, and the arithmetic is in section 1.3.4.

1.3.3 Threshold estimates

Wrong Concept 1.2: The threshold is a fixed number, about one percent

One percent is a numerical estimate for one code family, the surface code, under one noise model, independent depolarizing noise on every location (each qubit suffers X, Y, or Z with equal probability), and one architecture, nearest-neighbour gates on a two-dimensional lattice with fast measurement. For concatenated codes under generic noise the estimates are one to two orders of magnitude lower; for the same surface code under the idealized model in which only data qubits fail and syndromes are read perfectly, the estimate is about ten percent. A hardware platform does not have a threshold; a scheme does, and the question for a platform is whether its actual error rates, under its actual noise, fall below the threshold of a scheme its connectivity can implement.

The following values are approximate, quoted to the nearest order of magnitude, and depend on the noise model and architectural assumptions stated. They make one point: the word “threshold” covers numbers that differ by a factor of a thousand.

Scheme	Noise model and assumptions	Approximate threshold
Concatenated seven-qubit code	generic independent noise, rigorous proof	10^{-5} and below
Concatenated seven-qubit code	generic independent noise, numerical estimate	10^{-4} to 10^{-3}
Concatenated codes with heavy postselection	independent depolarizing noise, very large overhead	up to a few times 10^{-2}
Surface code	independent depolarizing noise on every location, nearest-neighbour gates	about 10^{-2}
Surface code	only data qubits fail, syndromes read perfectly	about 10^{-1}

The rigorous values are lower bounds obtained by counting every pair of locations that could cause failure; the numerical values come from simulations that track which pairs actually do. The last row is not achievable in any device, since it assumes perfect measurement, and is included only to show how strongly the number depends on what is assumed to be noisy. Correlated noise, leakage out of the two-level subspace, and drift of control parameters are outside all five rows and lower every figure.

1.3.4 Surface code overhead

A surface code of distance d stores one logical qubit in d^2 data qubits and uses about $d^2 - 1$ further qubits to measure its checks, about $2d^2$ physical qubits in all. Numerical studies under independent depolarizing noise [8] find a logical error rate per round of roughly $p_L \approx (p/p_{\mathrm{th}})^{(d+1)/2}$ up to a prefactor of order one; the exponent is the smallest number of errors a distance-d code can misidentify. At $p = p_{\mathrm{th}}/10$ each increase of d by two lowers p_L tenfold. A rate of 10^{-9} then needs $d \approx 17$ and about 600 physical qubits per logical qubit; a rate of 10^{-15} needs $d \approx 29$ and about 1700. A computation of 10^9 logical operations on 10^3 logical qubits, the scale at which the algorithm of **??** threatens deployed cryptography, therefore requires on the order of 10^6 physical qubits before counting distillation, which in current designs is comparable to or larger than the count for the data. At $p = p_{\mathrm{th}}/3$ the required distances roughly double and the qubit counts roughly quadruple. All figures are approximate and quoted for one noise model.

> **Remark 1.3** Limits of the theorem
>
> The theorem states that an ideal circuit can be simulated reliably at polylogarithmic overhead. It does not state that the overhead is small in absolute terms; for the algorithms of **??** on cryptographically relevant inputs, the dated estimates recorded in **??** give physical qubit counts of order one million. It does not state that the ideal circuit does anything useful; reliability is a property of the execution, and the question of quantum advantage belongs to **??????**. And it does not describe an existing machine. Experiments published in 2024 demonstrated a surface-code memory whose logical error rate decreased by a factor of about two each time the distance was raised from three to five to seven [9], the first clear demonstration of operation below threshold in hardware, but no fault-tolerant computation of useful size has been carried out. The theorem is the reason to believe such a computation is possible in principle, not evidence that one exists.

1.4 Summary

A fault is a failing component and an error is its effect on the data; one fault may cause several errors because a CNOT copies a bit flip from control to target and a phase flip from target to control. A correction circuit with a shared ancilla or a poorly ordered encoding can turn a single fault into an uncorrectable error and fail at first order in p, no better than no code at all. Fault tolerance is the requirement that a single fault anywhere in an encoded operation produce at most one error per block; it is achieved by transversal gates, verified ancillas, repeated syndrome measurement, and ancillas with the same connectivity as the data. Transversal gates alone are never universal, and the missing gate is supplied by distillation or code switching at considerable cost. The threshold theorem states that, under independent or weakly correlated noise with fresh ancillas, parallel gates, and fast classical control, a physical error rate below a scheme-dependent threshold allows the logical error rate to be made arbitrarily small at polylogarithmic overhead. The threshold is not one number, and the theorem is not a promise of useful computation.

The course has followed a single thread. **??** defined a state as a vector of amplitudes and a measurement as a choice of basis, and showed that relative phase is physical because interference makes it visible. **??** showed that the amplitudes of several qubits do not factor and that entanglement is a resource without classical counterpart. **??** gave a finite gate set from which every unitary can be approximated arbitrarily well. **??????** arranged those gates so that interference makes the final measurement answer a question with fewer queries than any classical method of the same kind, exact or bounded-error, and, for factoring, faster than any known classical method, and **??** showed what entanglement and measurement allow between distant parties. **??** showed that a real device leaks amplitude information into its environment and that a code can recover from a single error provided the recovery itself is reliable. This lecture has shown what “reliable” must mean and at what price it can be had. The result is a complete account of what a quantum computer is in principle: physical qubits, a universal gate set, an interference-based algorithm, and a fault-tolerant scheme operating below its threshold. Whether the last of these can be built at the scale the algorithms require is the open engineering question of the field.

Homework 4 is due at this session; no homework depends on material first introduced here.

References

[1] Peter W. Shor. Fault-tolerant quantum computation. In *Proceedings of the 37th Annual Symposium on Foundations of Computer Science*, pages 56–65, 1996.

[2] Andrew M. Steane. Error correcting codes in quantum theory. *Physical Review Letters*, 77: 793–797, 1996.

[3] Bryan Eastin and Emanuel Knill. Restrictions on transversal encoded quantum gate sets. *Physical Review Letters*, 102:110502, 2009.

[4] Dorit Aharonov and Michael Ben-Or. Fault-tolerant quantum computation with constant error. In *Proceedings of the 29th Annual ACM Symposium on Theory of Computing*, pages 176–188, 1997.

[5] Emanuel Knill, Raymond Laflamme, and Wojciech H. Zurek. Resilient quantum computation. *Science*, 279:342–345, 1998.

[6] Alexei Yu. Kitaev. Quantum computations: algorithms and error correction. *Russian Mathematical Surveys*, 52:1191–1249, 1997.

[7] Alexei Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303:2–30, 2003.

[8] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: towards practical large-scale quantum computation. *Physical Review A*, 86:032324, 2012.

[9] Google Quantum AI and collaborators. Quantum error correction below the surface code threshold. *Nature*, 638:920–926, 2025. arXiv:2408.13687.